Therefore, the word "bank" turns into the word "bank1", given by the equat" 1_"bank" + 0.11_"river". We are almost done. We want the coeffs of the words to add to 1, so we can divide everything by their sum, which is 1.11. When we do that, we get the the word "bank" gets transformed into the word

0.9 * "bank" + 0.1 "river".

↳ Let's do one more for consistency. The word "money" in the 2nd sentence turns into "money2", given by the eq" 1 * "money" + 0.25 * "bank". When we normalize, we divide everything by 1.25, to get the equation

0.8 * "money" + 0.2 * "bank".

This is the way we obtained the modified words that we use in the attention mechanism above. The modified sentences then become the following:

original sentence

the banks of the river

⇓

Modified sentence

the 0.9 bank + 0.1 river of the 0.1 river + 0.9 bank

original sentence

money in the bank

⇓

Modified sentence

0.8 money + 0.2 bank in the 0.2 money + 0.8 bank

⇒ Is there more

What you learned

However, we can do

method called mult

does not only

different ones. T

original by t

multi-head atten

at the task

[Check 2 vid

(LLMs make grea

in order to

⇒ Attention helps

of the text as oppo

So, Attention does

be seen geometrically

computer because hum

understands numbers.

models are, because

each word or eac

easier.

⇒ Is there more to Attention ?

What you learned in this chapter is simple self-attention. However, we can do much better than that. There is a method called multi-head attention, in which one does not only consider one embedding but several different ones. These are all obtained from the original by transforming it in different ways. Multi-head attention has been very successful at the task of adding context to text.

[Check 2 videos]

(LLMs make great use of attention mechanisms in order to understand text)

⇒ Attention helps the LLM understand the whole context of the text as opposed to just a few words at a time. So, Attention does a few mathematical operations that can be seen geometrically. Embeddings are bridge b/w human & computer because human can understands text & computer can understands numbers. Better the embedding, better the models are, because if you have really good numbers for each word or each piece of text then prob^m becomes much easier.

5) Attention < self Attention / multi-head "

↳ Attention uses the context of the sentence in order to help the embedding resolve this type of ambiguity.

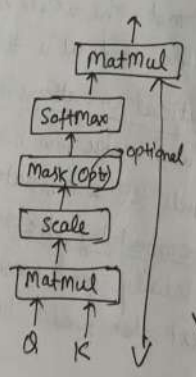
↳ Attention can be explained using below formulas using matrices such as Query, Key & Values.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

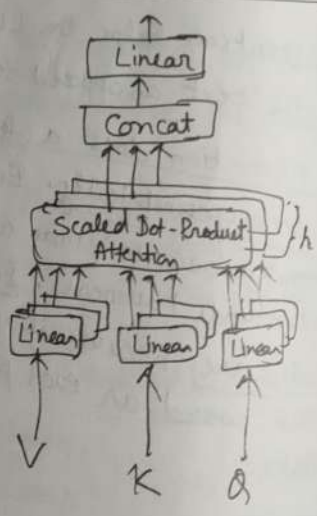
$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W^O$$

$$\text{where head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Scaled Dot-Product Attention



Multi-Head Attention



⇒ Example

① Please bu

② Apple
(Brand)

moved Ora

more apple
more apple
Phone

↳ At the end of
the context.

ce in order to
ambiguity

formulas
ues.

$T) V$

$h) W^0$

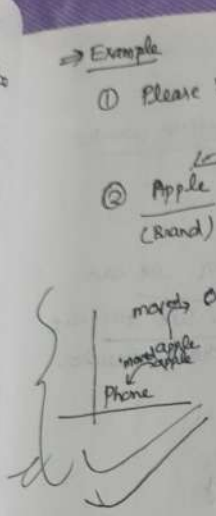
$W_i^q, KW_i^k,$
$VW_i^v)$

Attention

⇒ Example

① Please buy an apple (Fruit) and an orange

② Apple (Brand) unveiled the new phone



Say, initially orange has coordinate (11,11)
& Phone " " (1,2)
& Apple " " (6,7)

Now, For modified sentences,
apple has co-ord. (8, 8.5) in 1st sentence
& apple has coord (4,5) in 2nd sentence.

If we use these new numbers (vectors) as embeddings in the sentence then model is going to have an easier time

↳ This process is done many times in ML, so it becomes nearer to relevant region

↳ At the end of the day, we don't use sentence, we use context.

→ Words pull each other like gravity

⇒ Multi-head Attention

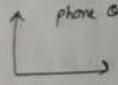
↳ Attention works better but multi-head attention works even better.

↳ Is one embedding good enough ?

No, we can do much better. We can ideally have lots & lots of embeddings. We just use 10, 20 or 100 embeddings & combine those results. That is Multi-head Attention.

Ex.

= good embedding



= worst embedding.

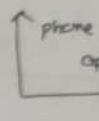
↳ We build embedding by modifying existing embedd with the use of Linear transformations.

{ rotation, stretching (horizontal, vertical)
shear

⇓

we can combine these transf

Ex. Get new



does

&

Now,

embedd

①

②

③

↳ We get

& we t

score. &

→ Words pull each other like gravity

⇒ Multi-head Attention

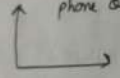
↳ Attention works better but multi-head attention works even better.

↳ Is one embedding good enough ?

No, we can do much better. We can ideally have lots & lots of embeddings. We just use 10, 20 or 100 embeddings & combine those results. That is Multi-head Attention.

Ex.

= good embedding



= worst embedding.

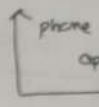
↳ We build embedding by modifying existing embedd with the use of Linear transformations.

(rotation, stretching (horizontal vertical)
shear

⇓

we can combine these transf

Ex. Get new



does

&

Now,

embedd

①

②

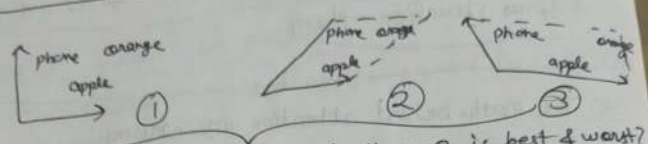
③

↳ We get

& we t

score. &

Ex. Get new embeddings from existing ones



which one of these 3 is best & worst?

↳ 3rd one is best because it separates phone & orange very well & 2nd one is worst because it doesn't separates phone & orange well. & 1st one is okay.

Now, if we want to give score these embeddings, then,

① ⇒ score = 1
② ⇒ score = 0.1
③ ⇒ score = 4

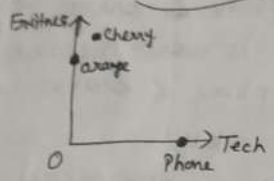
} random nos.

↳ We get these scores when we train neural N/W & we train these embeddings & we train these score. & so, key, query & value matrix comes.

↳ The K & Q matrices are going to help us build a bunch of transformations & V are going to help us visualize them.

(*) The math behind attention mechanisms

Measure 1 ; Dot Product → can be negative



	Tech	Fruitness
Cherry	1	4
orange	0	3

} coordinate

↳ Similarity $\boxed{1*0+4*3} = 12$

↳ Similarity b/w Cherry & Phone
(1,4) (3,0)
$1 \cdot 3 + 4 \cdot 0 = 3$

↳ Similarity b/w orange & Phone
(0,3) (3,0)
$0*3+3*0=0$

orange & Phone are Perpendicular w.r.t origin.

Measure 2; Cosine Similarity. (b/w -1 & 1) → very similar, → very diff.

Similarity (cherry, orange) = $\cos 14° = 0.97$

" (cherry, phone) = $\cos 76° = 0.24$

" (orange, ") = $\cos 90° = 0$

⇒ If all vectors are norm = 1 then

Dot product = Cosine similarity

Measure ③ : Scaled dot product

Dot product divided by the square root of the length of the vector.

↳ It is used in attention.

So, Similarity (cherry, orange) = $\frac{1*0+4*3}{\sqrt{2}} = \frac{12}{\sqrt{2}} = 8.49$

(1,4) (0,3)

because vector has 2 coordinates so, length = 2

Similarity (cherry, Phone) = $\frac{1*3+4*0}{\sqrt{2}} = 2.12$

(1,4) (3,0)

Similarity (orange, Phone) = $\frac{0}{\sqrt{2}} = 0$

(0,3) (3,0)

↳ We are dividing by square-root of length of vector because if we have very long vectors (say 1000 coordinates) then we get very-2 large dot product & we don't want & we want to manage it. So, we divide by say $\sqrt{1000}$.

...ample

Sentence ① an apple and an orange

Sentence ② an apple phone.

& we have 3 coordinates for each as..

	Tech	Fruitness	Other
Orange	0	3	0
Phone	4	0	0
Apple	2	2	0
And	0	0	2
An	0	0	3

Now, Cosine similarity matrix.

	Orange	Phone	Apple	And	An
Orange	1	0	0.71	0	0
Phone	0	1	0.71	0	0
Apple	0.71	0.71	1	0	0
And	0	0	0	1	1
An	0	0	0	1	1

Now, see the same table for Sentence ①

	Orange	Apple	And	An	
Orange	1	0.71	0	0	→ 1 * orange + 0.71 * Apple
Apple	0.71	1	0	0	→ 0.71 * Orange + 1 * Apple
And	0	0	1	1	→ 1 * And + 1 * An
An	0	0	1	1	→ 1 * And + 1 * An

Linear Combinat^n

For Sentence ②

	Phone	Apple	An	
Phone	1	0.71	0	→ Phone = 1 Phone + 0.71 Apple
Apple	0.71	1	0	→ Apple = 0.71 Phone + 1 Apple
An	0	0	1	→ An = 1 ~~An~~

↳ Normalizat^n.

Sentence① Orange = $\frac{1 \text{ orange} + 0.71 \text{ Apple}}{1+0.71}$ = 0.58 Orange + 0.42 Apple

But there is a Prob^m

If we have Orange = $\frac{1 \text{ Orange} - 1 \text{ Motorcycle}}{1-1 = 0 \ ?}$

So, need Coeffs to be positive.

Sol^n. Coeff → e^{Coeff}.